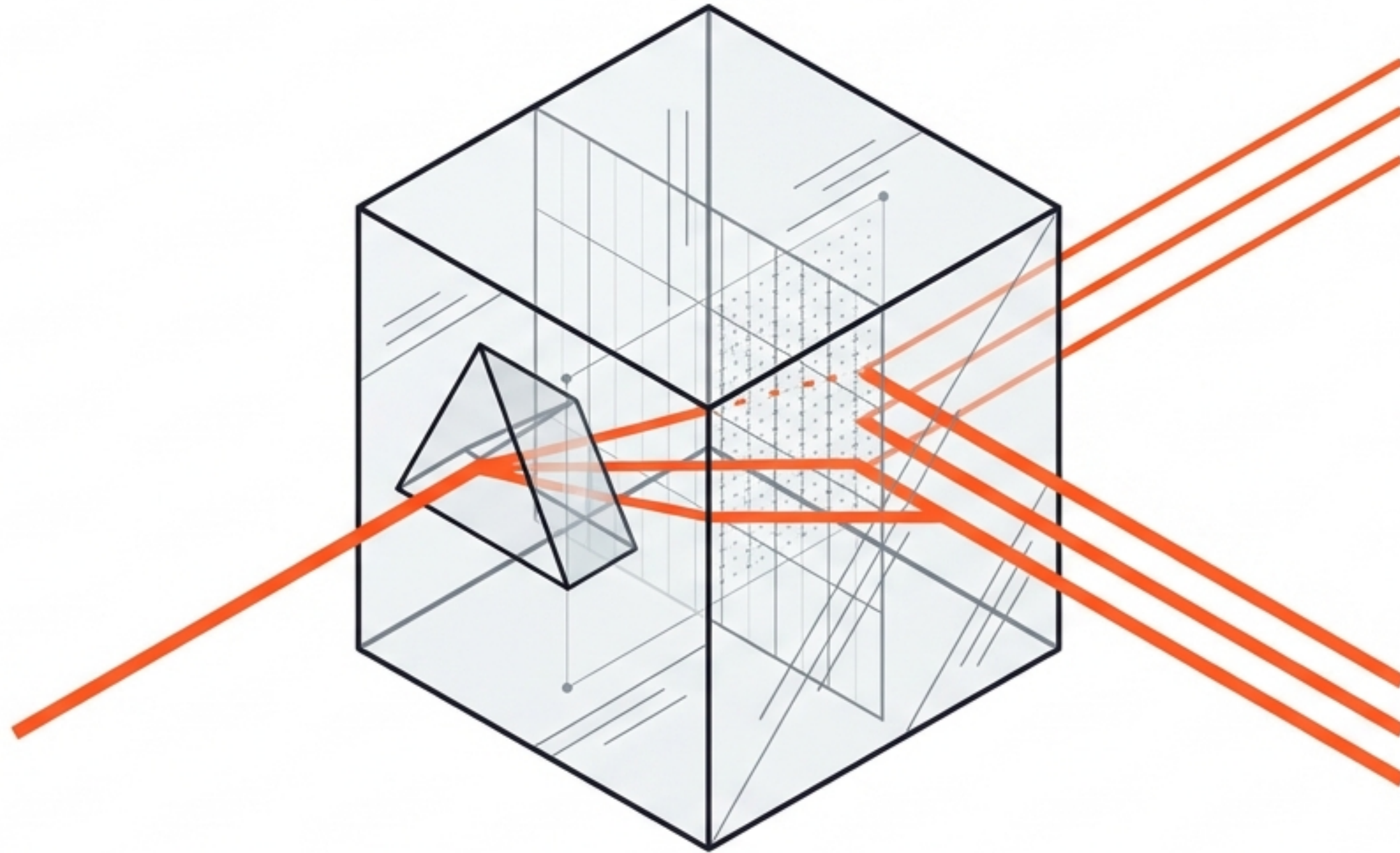


# MITM Proxy Platform: Solution Architecture Debrief

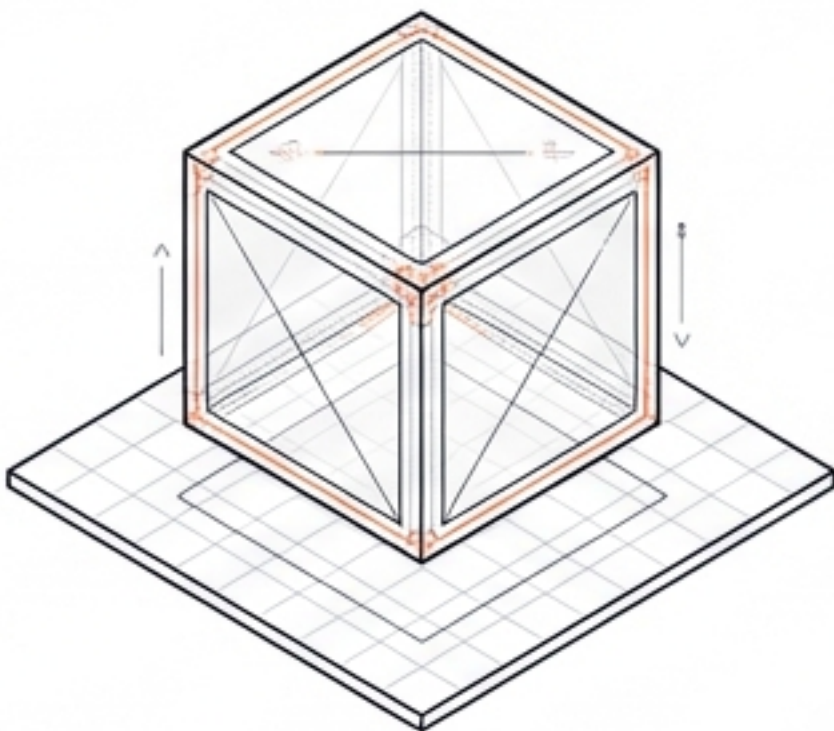
Real-time Traffic Interception, Processing, and Content Intelligence



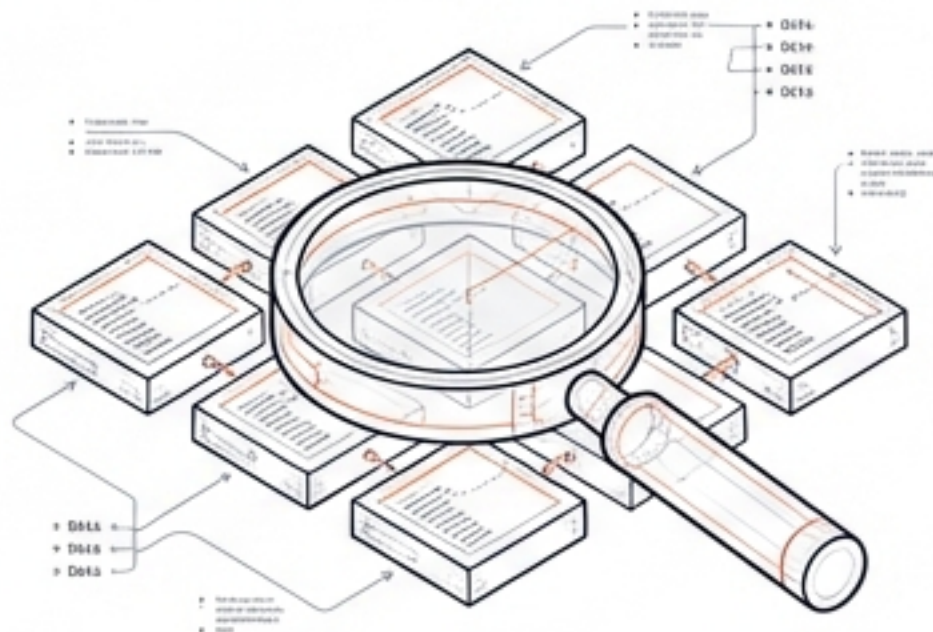
PREPARED FOR TECHNICAL STAKEHOLDERS AND BUSINESS PARTNERS



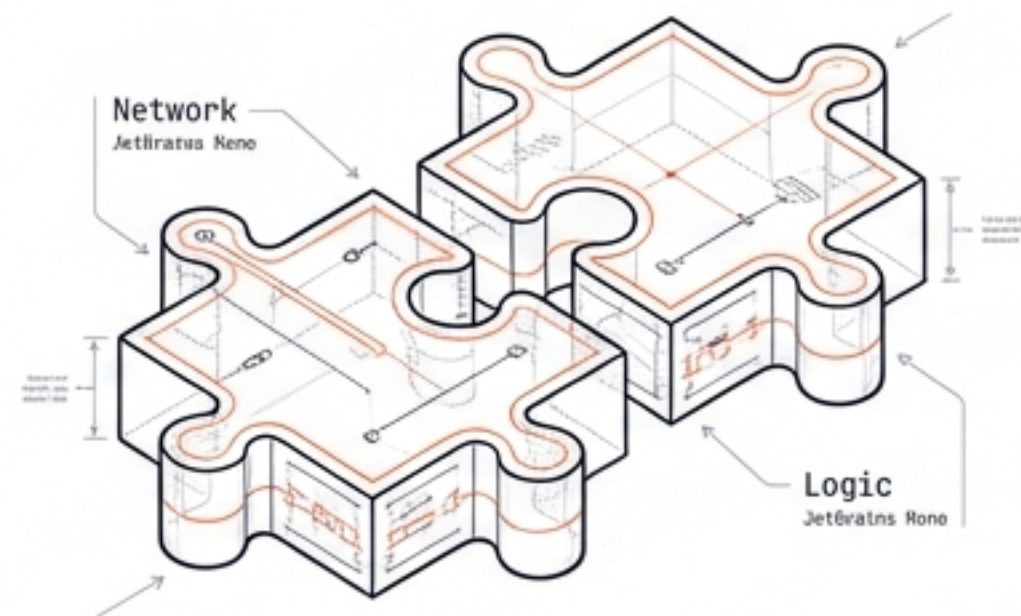
We have developed a Man-in-the-Middle (MITM) platform designed to intercept HTTP(S) traffic transparently between the client browser and the internet. This creates a strategic control point for reliability and intelligence.



Requires no modification to client or server-side applications. The proxy is transparent to the end-user workflow.



Enables advanced analysis, caching, and transformation capabilities. We treat web traffic as a programmable dataset.

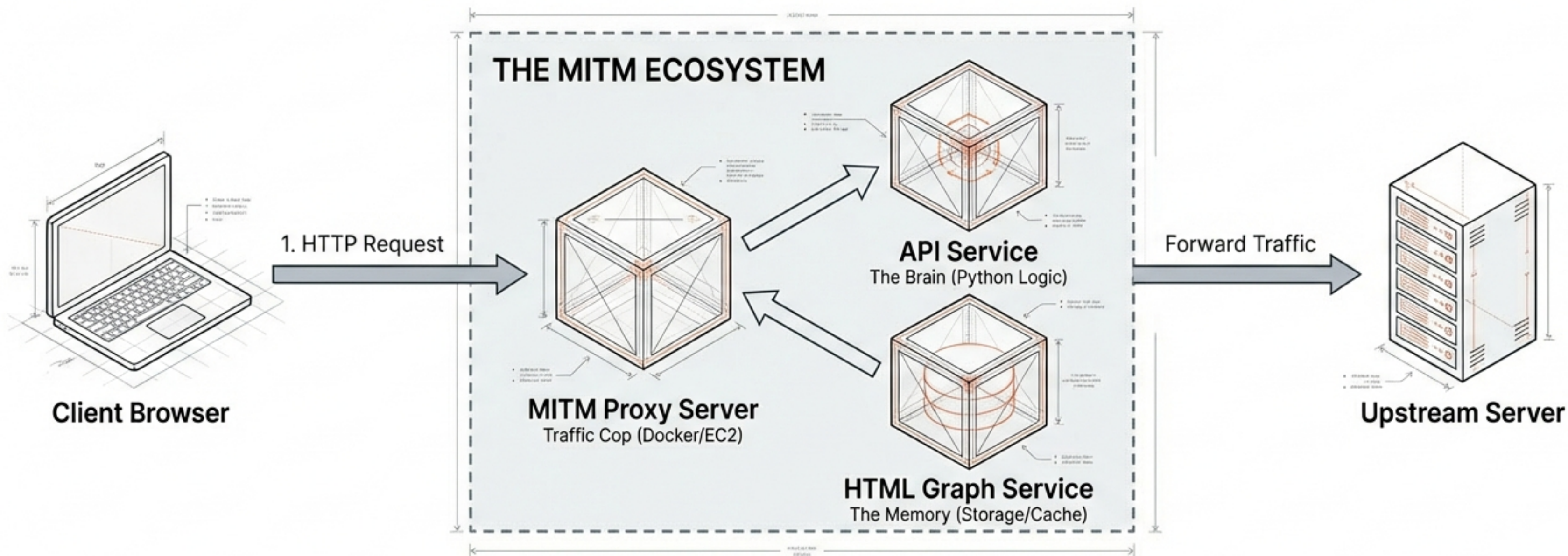


Decouples low-level network traffic handling from high-level business logic, ensuring maximum stability and distinct separation of concerns.

**Key Takeaways:** A battle-hardened system built on proven proxy technologies, laying the future-proof foundation for 'HTML Graph' intelligence.



# High-Level System Architecture and Component Interaction



## Client Perspective

Trusts the proxy's certificate; sees the proxy as the authoritative server.

## Separation of Duties

The Proxy handles the network I/O; the API handles the decision logic.

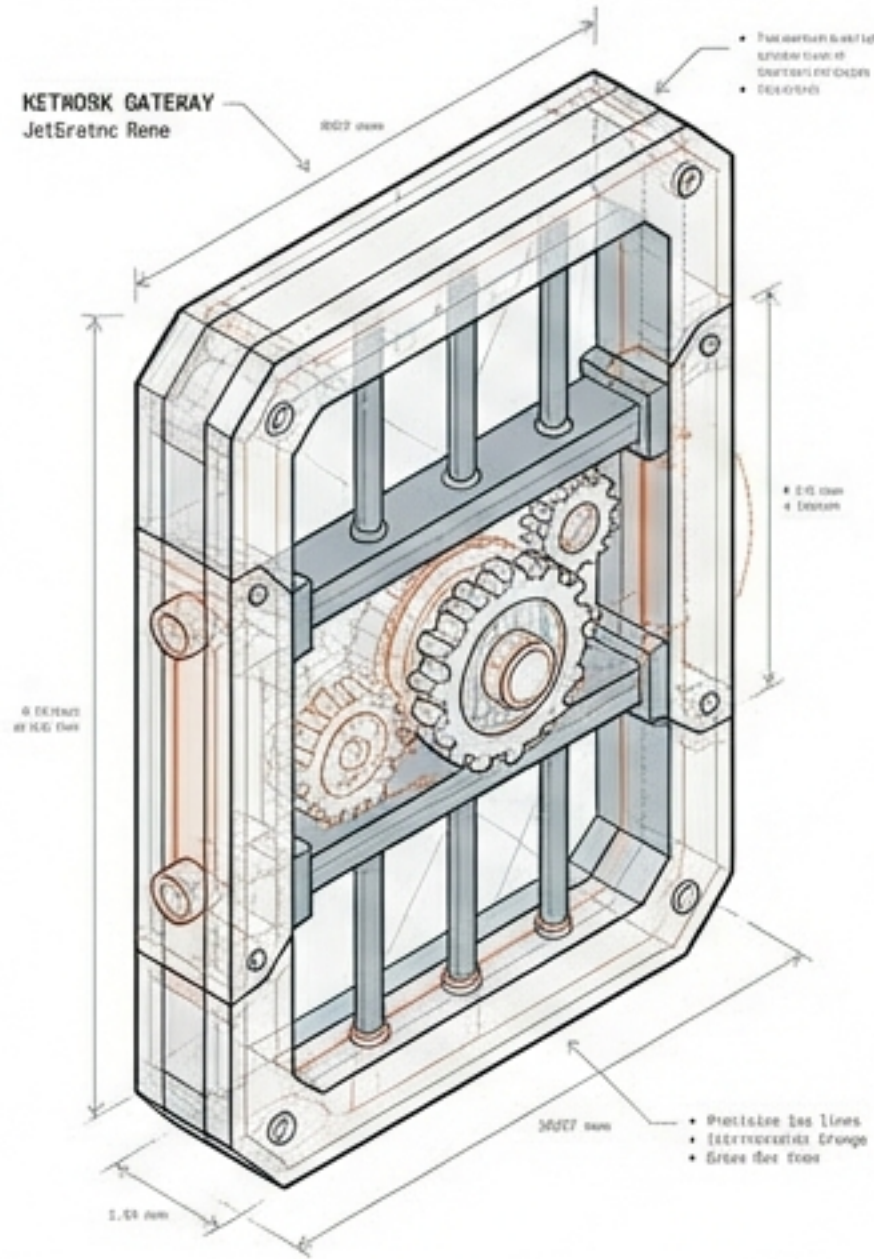
## Upstream Perspective

Sees the proxy as the client. Unaware of interception.



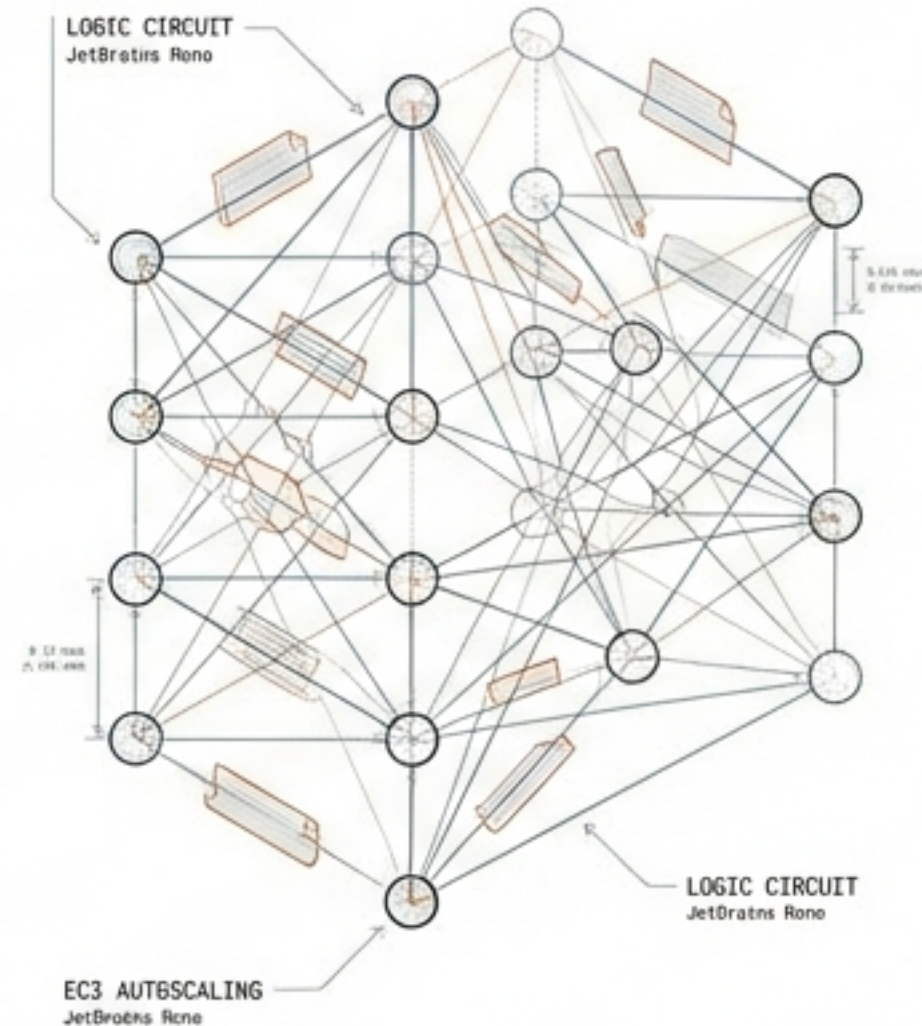
# Component Deep Dive: Decoupling Network from Logic

## The MITM Proxy Server



- **Role:** Interceptor & Traffic Cop.
- **Characteristics:** Stateless, battle-hardened, high-performance networking.
- **Function:** Pauses traffic at two hook points—Request and Response—and delegates decisions.
- **Scalability:** Horizontal scaling on EC2 behind a Load Balancer.

## The MITM Proxy API Service

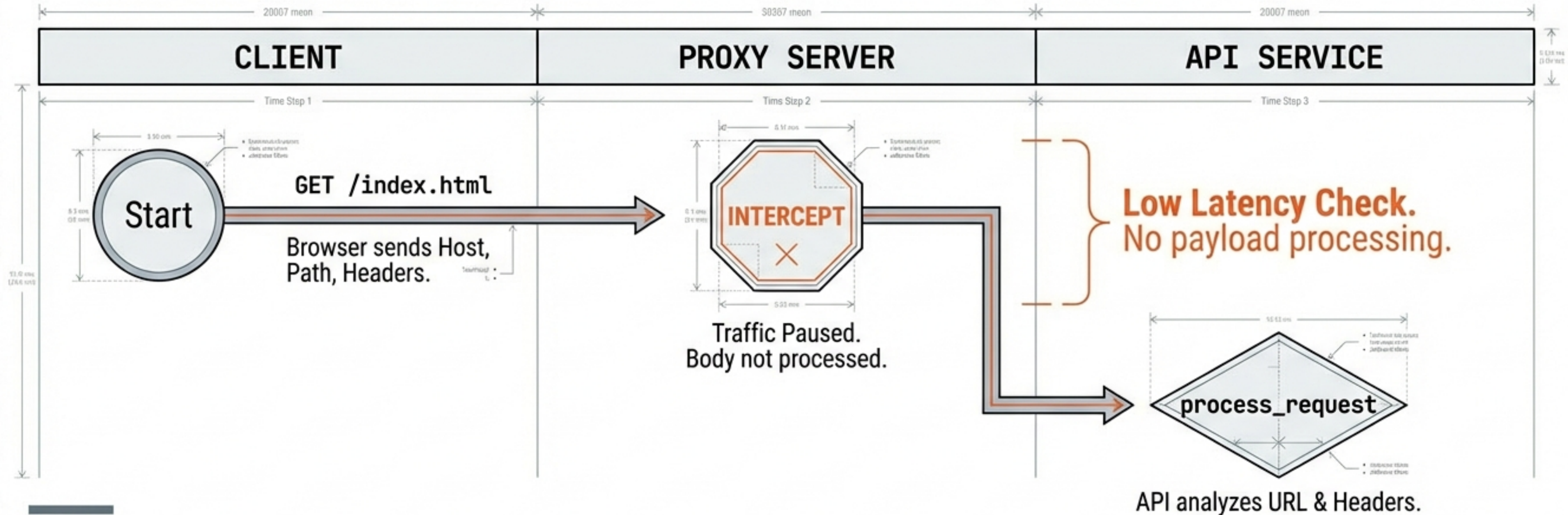


- **Role:** Decision Maker & Brain.
- **Language:** Python-based logic service.
- **Logic:** Evaluates headers to decide: Serve from cache? Log flow? Modify headers?
- **Versatility:** The API is unaware if the call comes from a real proxy or the Simulator.



# Phase 1: Request Interception and Analysis

## The Journey of a Packet – Part 1

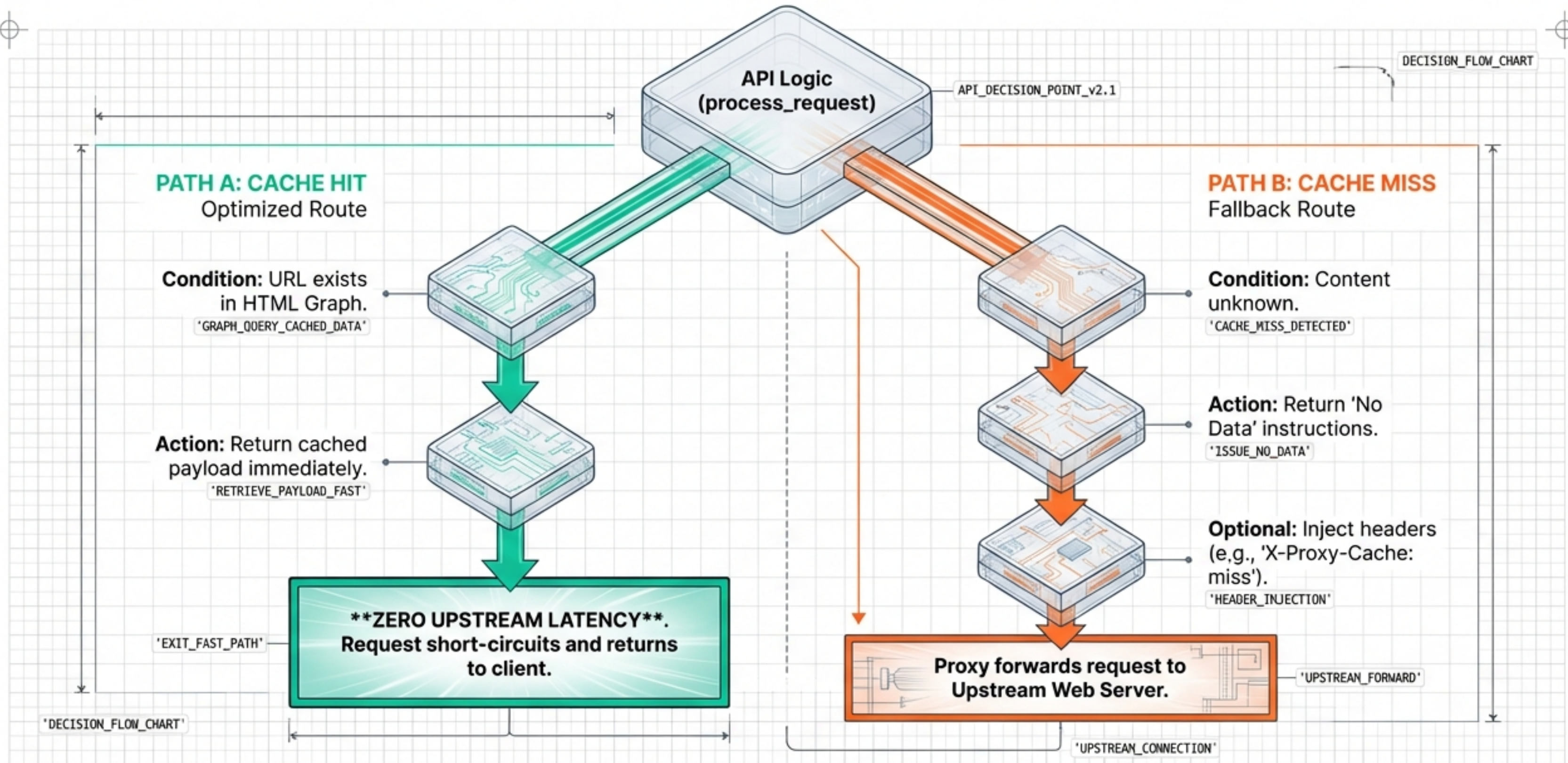


### Key Insight

The system prioritises speed. It only analyses the URL and headers necessary for caching decisions, avoiding heavy payload processing at this stage.



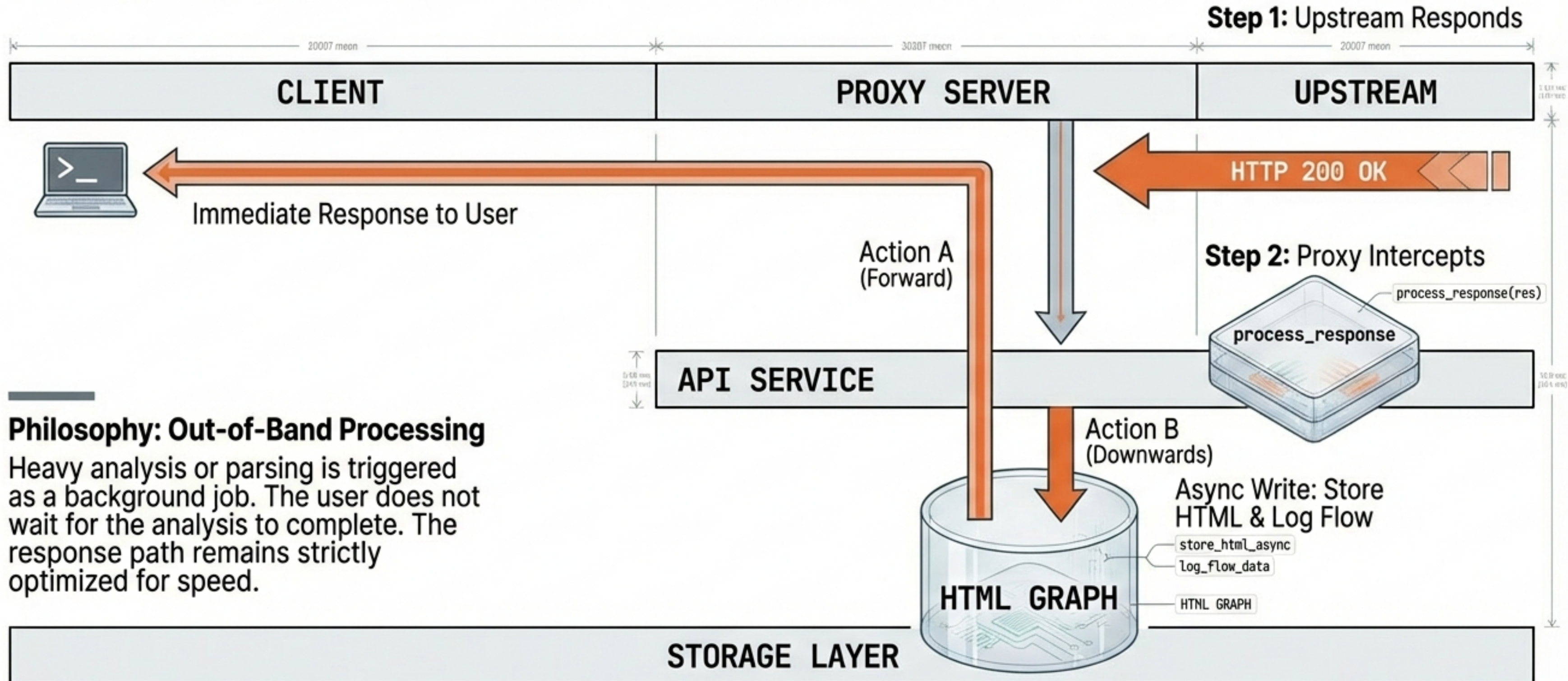
# The Decision Engine: Cache Hit vs. Cache Miss





# Phase 2: Response Capture and Out-of-Band Processing

## The Journey of a Packet – Part 2 (The Return)

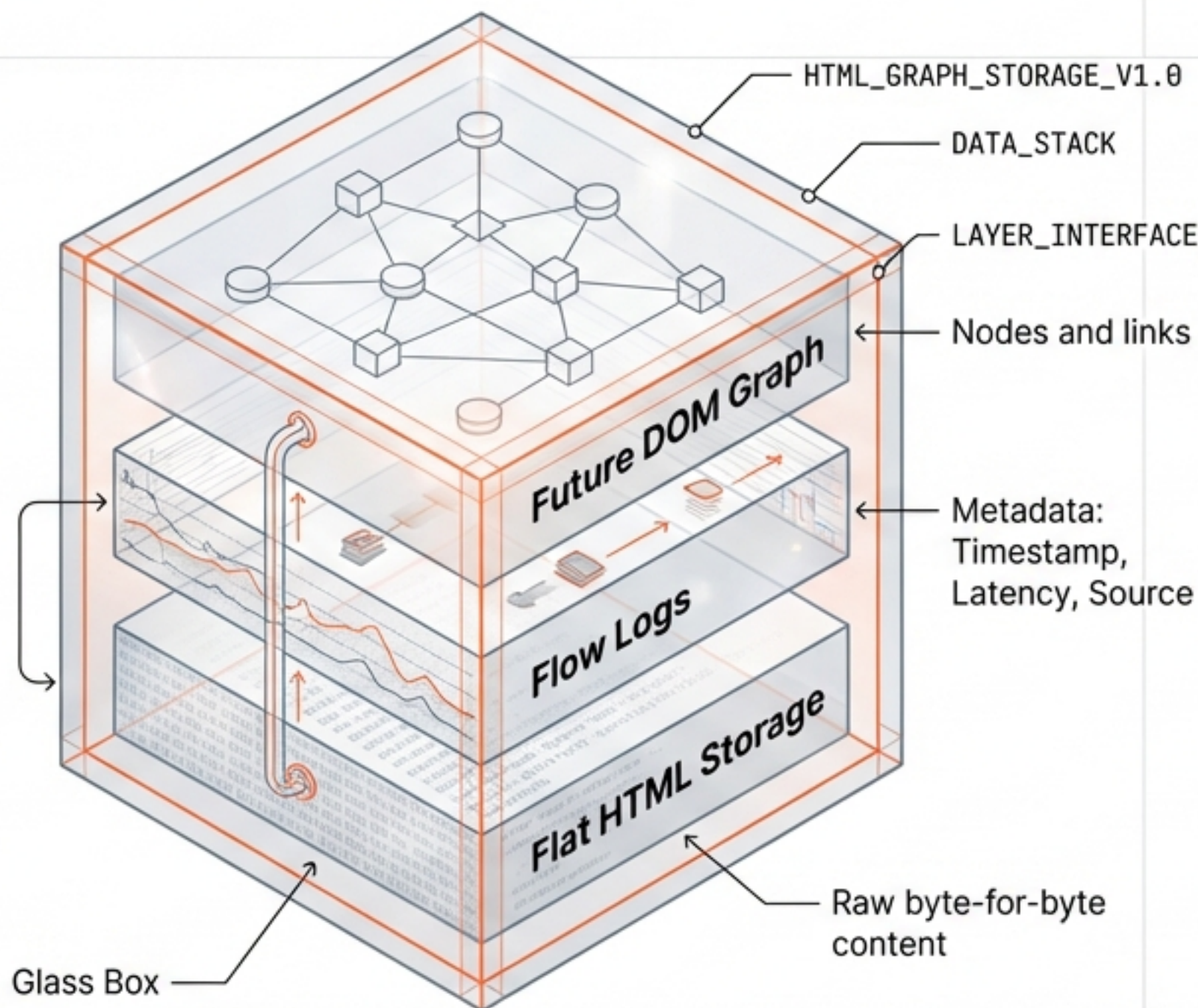


### Philosophy: Out-of-Band Processing

Heavy analysis or parsing is triggered as a background job. The user does not wait for the analysis to complete. The response path remains strictly optimized for speed.



# The Intelligence Layer: HTML Graph Service



- **Storage Architecture**

Currently stores raw HTML keyed by URL to ensure identical delivery on cache hits.

- **The 'Flow' Concept**

Every action creates a historical record.

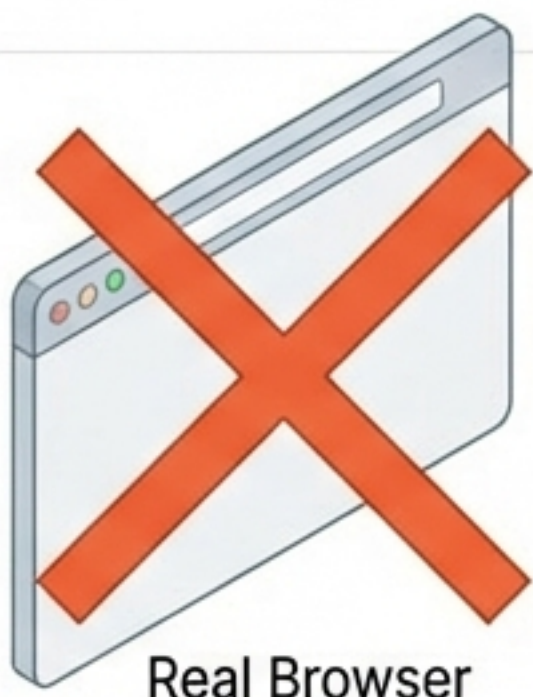
```
'Fetched 10:00:01 | Source: Upstream |  
Duration: 200ms'
```

- **Future Vision**

Moving from flat files to a queryable graph of DOM nodes and linked resources.



# Simulation Framework and Costing Analysis



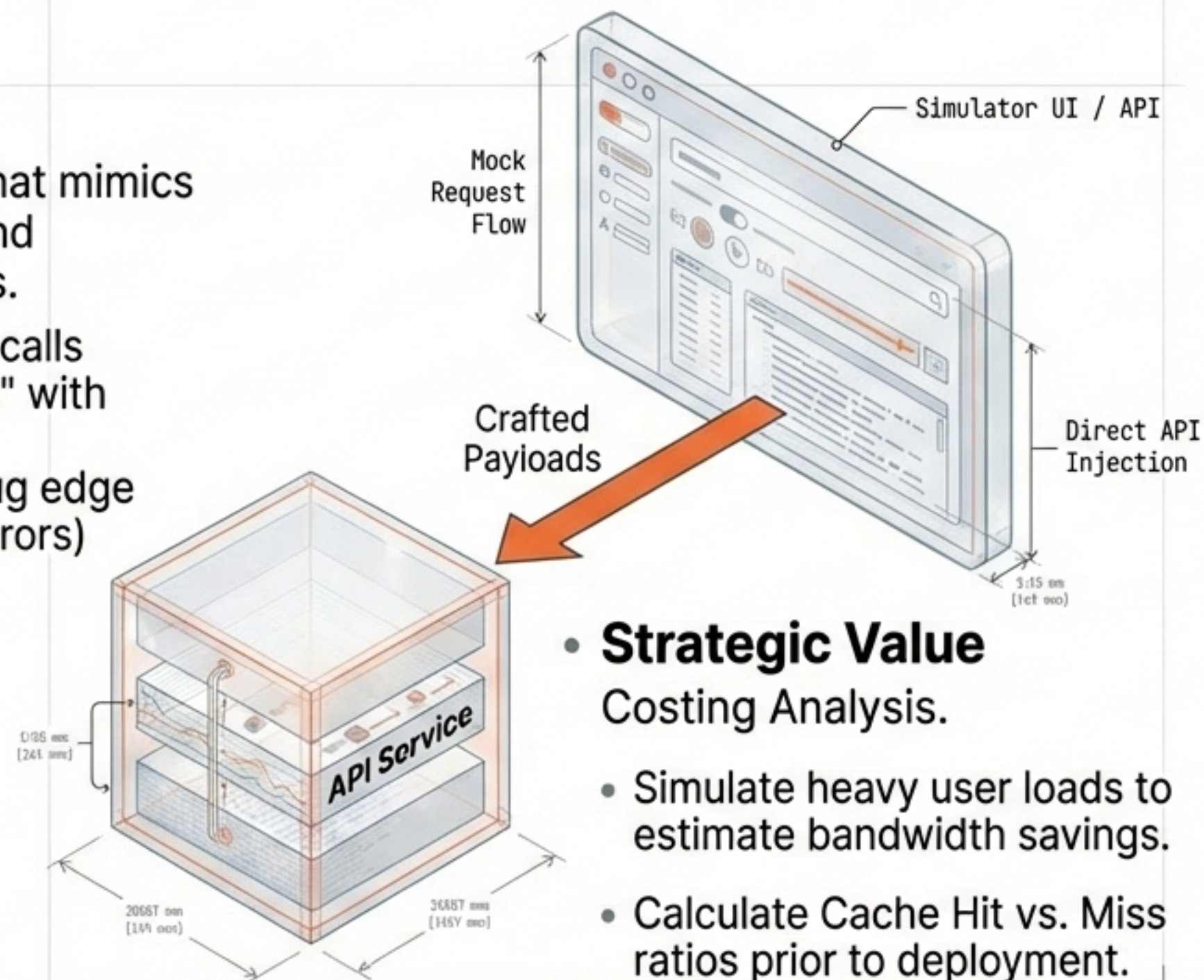
- **The Problem**

Testing with real browsers is slow, manual, and difficult to automate.

- **The Solution**

Custom Simulator that mimics browser requests and upstream responses.

- **Function:** Directly calls "process\_request" with crafted payloads.
- **Capabilities:** Debug edge cases (headers, errors) without network network traffic.



- **Strategic Value**

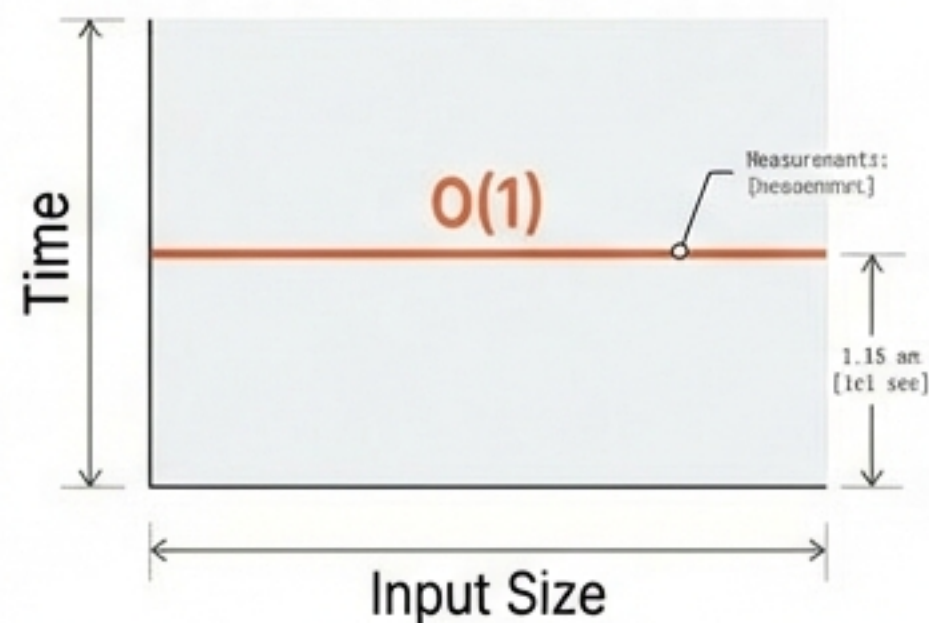
Costing Analysis.

- Simulate heavy user loads to estimate bandwidth savings.
- Calculate Cache Hit vs. Miss ratios prior to deployment.



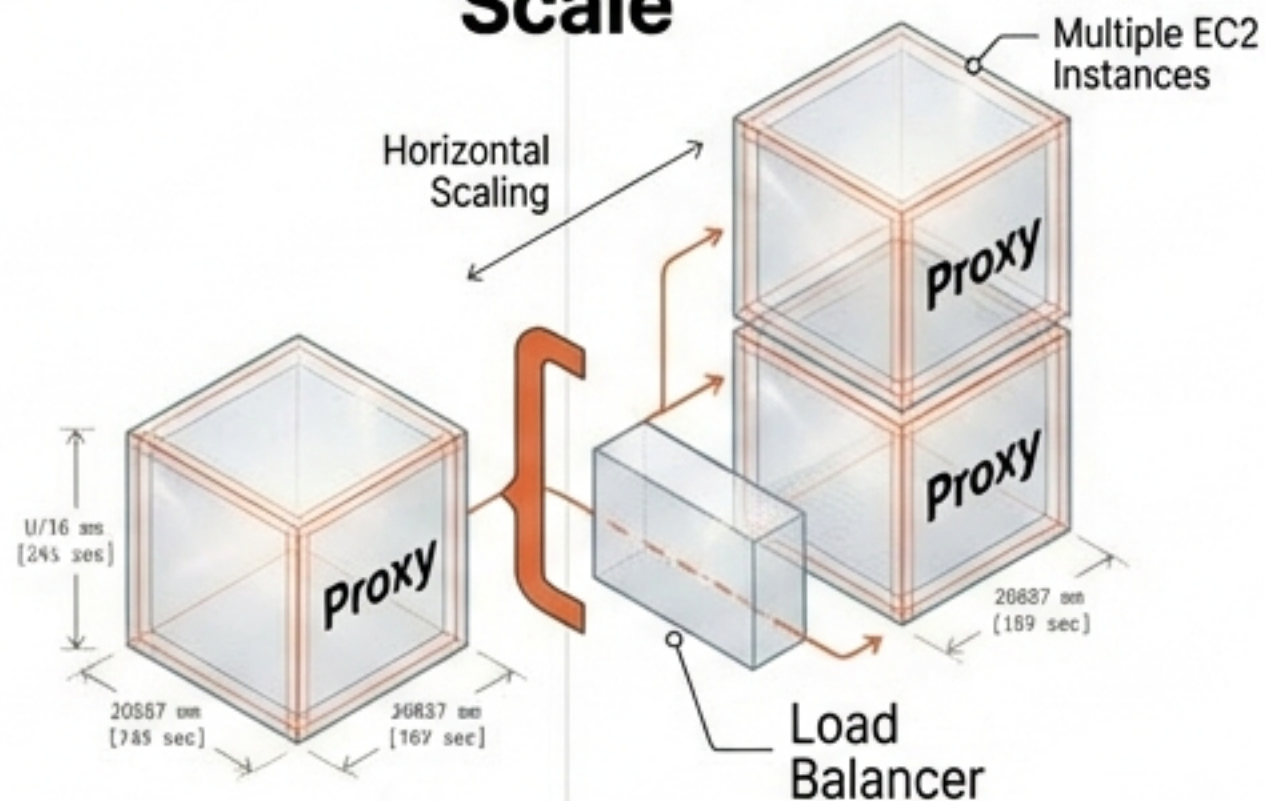
# Performance, Scalability, and Reliability

## Speed



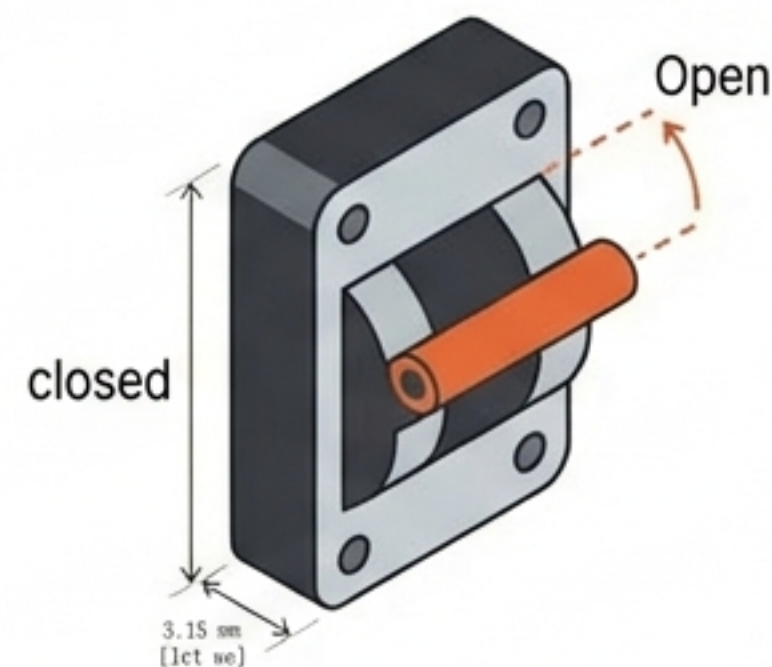
Cache lookups are constant time operations. Writes are asynchronous to prevent blocking the response stream.

## Scale



**Stateless Proxy Design:** Horizontal scaling with multiple EC2 instances.  
**Decoupled API:** Can be scaled independently.

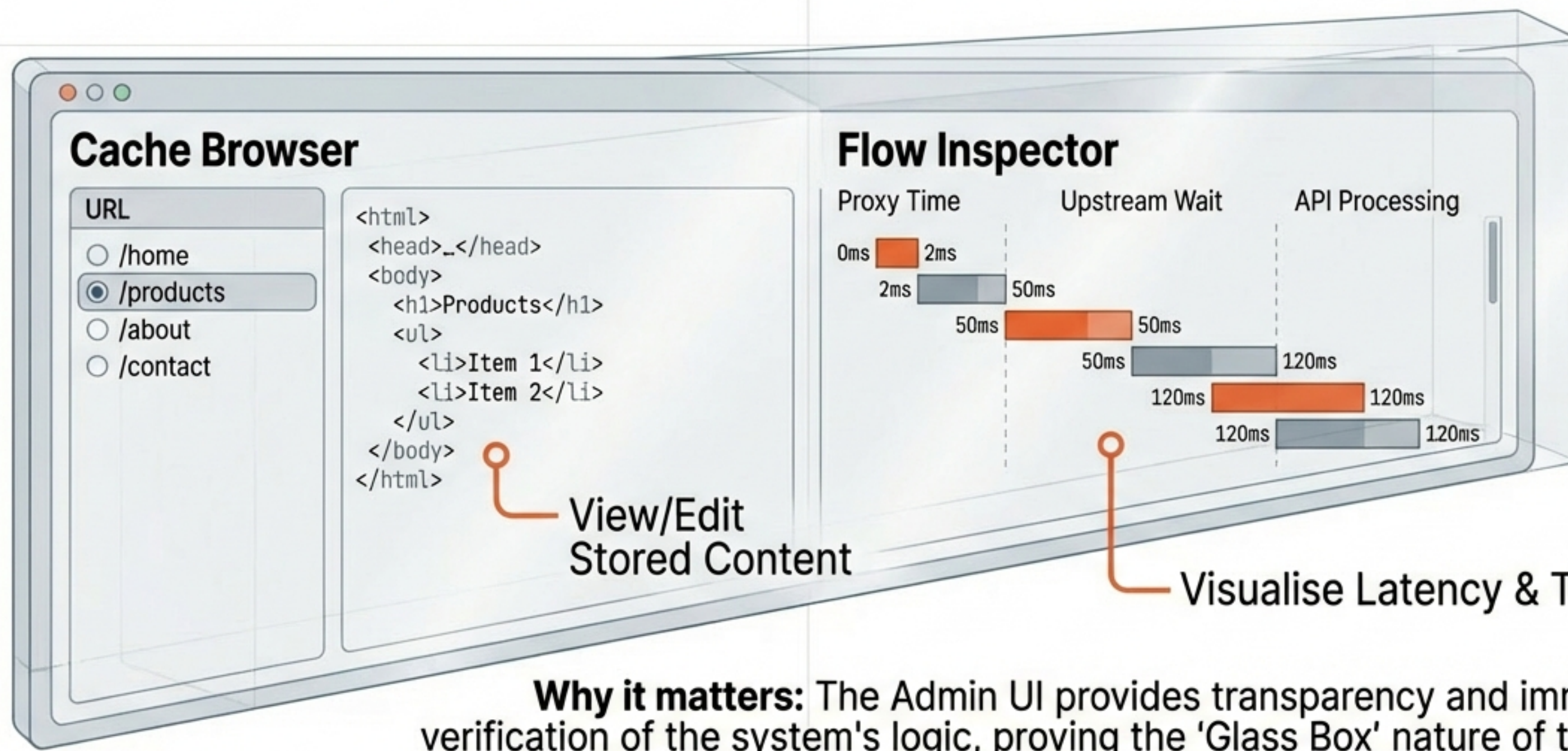
## Reliability



**Fail-Safe Strategy:** If the API hangs, the proxy falls back to direct forwarding. The user is never stalled by logic failures.



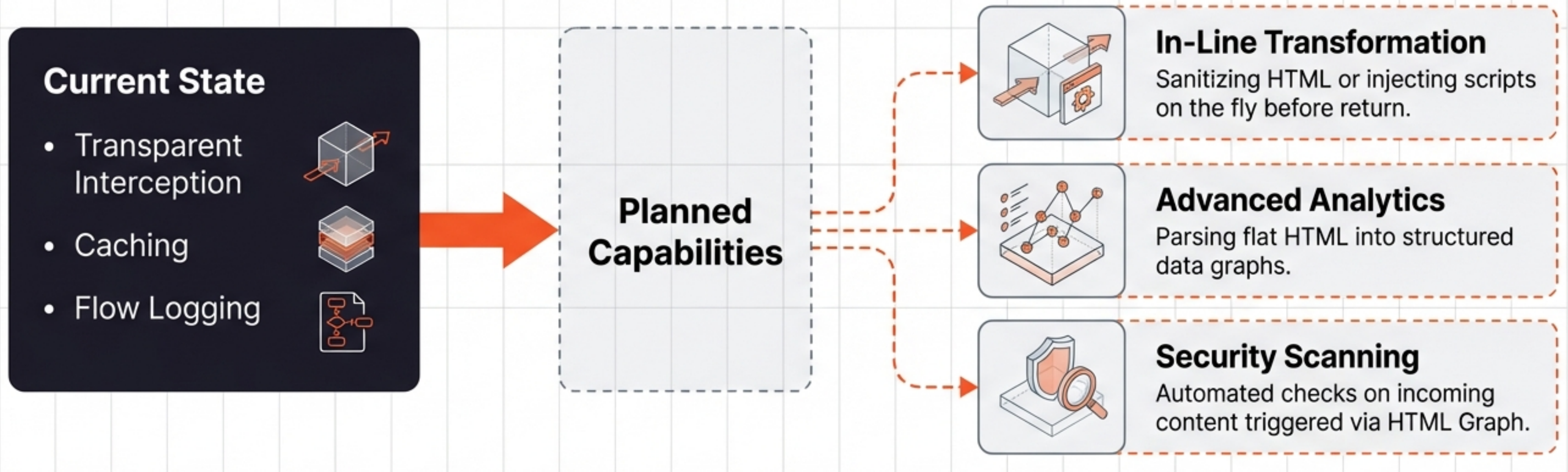
# Administration, Visibility, and Control



**Why it matters:** The Admin UI provides transparency and immediate verification of the system's logic, proving the 'Glass Box' nature of the tool.



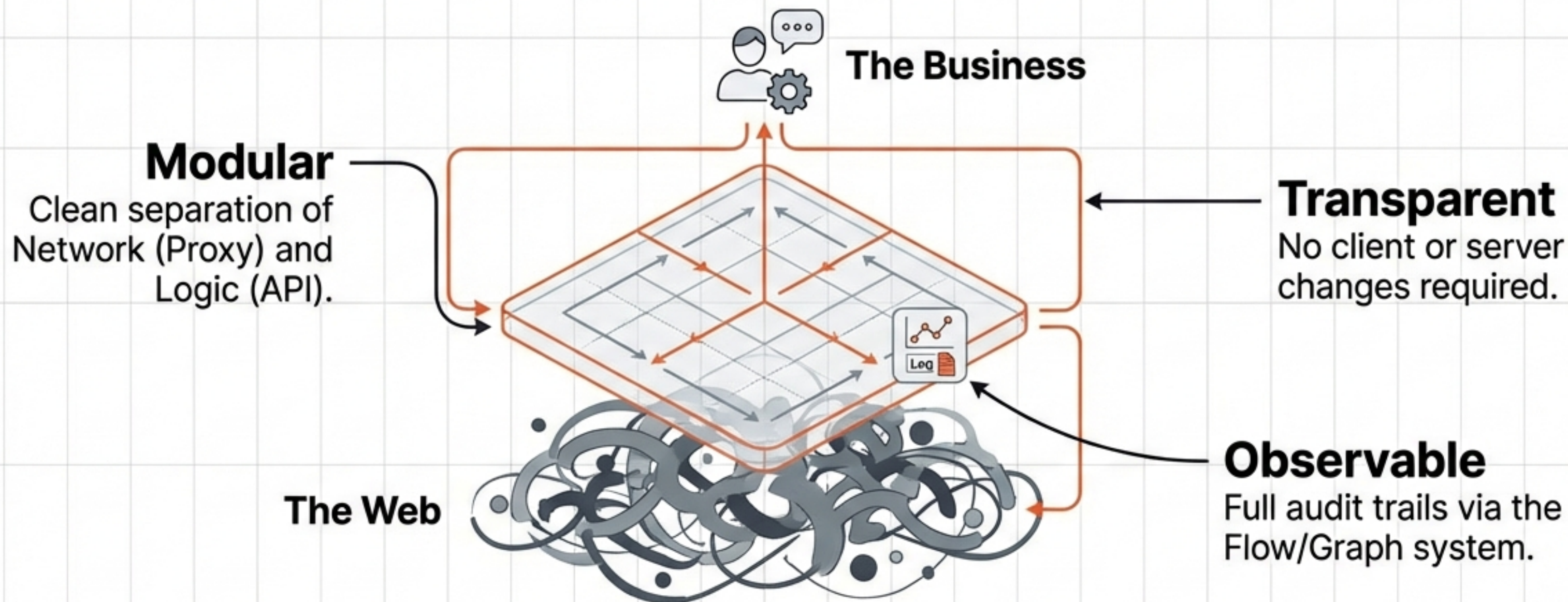
# Roadmap: Extensibility and Future Capabilities



**Strategy:** New processing flows can be added to the API logic with minimal impact on core proxy stability.



# Summary: A Foundation for Web Intelligence



The platform effectively creates a programmable control layer over the web. It enables optimization today while establishing the infrastructure for intelligent analysis and transformation tomorrow.